



ORACLE

MySQL Implementation Essentials Bootcamp

Installation and Architecture

Marco Carlessi (marco.carlessi@oracle.com)
Andreea Ciobotaru (andreea.Ciobotaru@oracle.com)
MySQL Sales Consulting

2023



Program Agenda

- Introduction
- Oracle MySQL Enterprise
- Oracle Cloud Infrastructure overview
- **Installation and Architecture**
- Database Design
- Security
- Backup
- Replication
- High Availability
- MySQL Enterprise Monitor
- HeatWave

The perfect MySQL Server

Some practical Tips



- Choice of Operating system: Linux or Windows or Solaris or ... ?
 - <https://www.mysql.com/support/supportedplatforms/database.html>
- MySQL in physical machines, virtual environments, containers or within Cloud?
 - MySQL is supported inside VMs and containers
- MySQL 8.0 scale very well on multiple cores
 - <http://dimitrik.free.fr/blog/posts/mysql-performance-over-18m-qps-with-80-ga-on-2s-skylake.html>
 - MySQL 8.0 benefit of multi-threading and NUMA
- InnoDB loves Memory (it cache data and keys)
 - Assign enough memory for the active data set + keys
- Fast disks preferred (SSD, FusionIO etc.)
- Exclude single point of failure

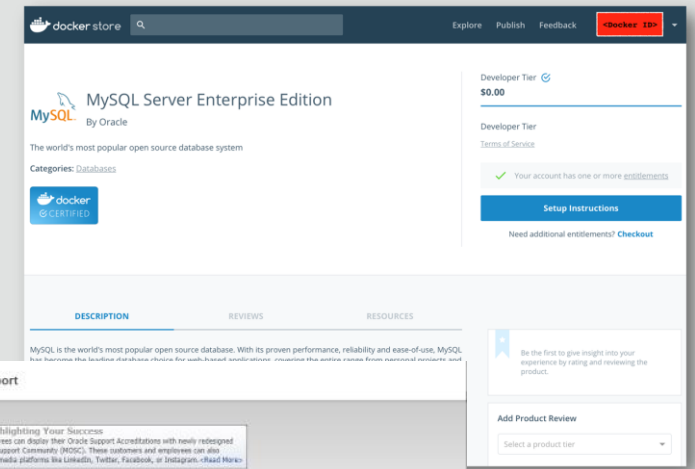
MySQL Downloads

- MySQL Community
 - <https://dev.mysql.com/downloads/>
- MySQL Enterprise Edition
 - <https://support.oracle.com>
- MySQL Trial
 - <https://edelivery.oracle.com>
 - Only **last release** of each MySQL product
- MySQL is opensource, source code is available
 - <https://dev.mysql.com/downloads/>
 - <https://github.com/mysql>



MySQL Enterprise and Docker

- MySQL supported inside virtualization or containers
 - [KB 1383964.1](https://dev.mysql.com/doc/refman/8.0/en/faqs-virtualization.html)
 - <https://dev.mysql.com/doc/refman/8.0/en/faqs-virtualization.html>
- MySQL can be integrated in docker images
 - MySQL Community available on Docker Hub
- MySQL Enterprise docker images available on
 - Docker Store
<https://store.docker.com/>
 - My Oracle Support
<https://support.oracle.com>
 - Oracle Container Registry
<https://container-registry.oracle.com>



MySQL Documentation

- Manuals (community and commercial)
 - <https://dev.mysql.com/doc/>
- Knowledge Base (partners & customers only)
 - <https://support.oracle.com>
- Blogs from MySQL engineers
 - <https://blogs.oracle.com/mysql/>
- Blogs moved to <https://dev.mysql.com/blog-archive/>
 - <https://insidemysql.com>
 - <https://mysqlservletteam.com/>
 - <http://mysqlhighavailability.com/>
- The MySQL Community evangelist
 - <https://lefred.be/>
- Performance expert
 - <http://dimitrik.free.fr>



MySQL – Installation

- Many Operating Systems are supported and documented, like
 - Unix/Linux
 - Microsoft Windows
 - OS X
 - Unbreakable Linux Network (ULN)
 - Solaris and OpenSolaris
 - FreeBSD2.9
- Documentation describes installation, post installation setup and test, upgrade and downgrade

<https://dev.mysql.com/doc/refman/8.0/en/installing.html>

MySQL – Installation

Windows

- Installation can be made with
 - MySQL Installer
 - Includes most MySQL Packages for Windows in one single package
 - Some MySQL apps rely on Microsoft .NET Framework 4.5.2 and Microsoft Visual C++ 2019 Redistributable Package
 - MySQL Database, MySQL Connectors, MySQL Workbench, mysqlslap (Load generation tool) and sample models
 - Documentation and Sample databases
 - First setup by wizard
 - Binary zip ("Noinstall Archive")
 - just the binaries
 - manual configuration

MySQL – Installation

Linux

- Installation can be made with repositories, packages (yum, rpm, deb) or binaries (tarball and zip)
 - There is no best option, it's a matter of
 - knowledge
 - company standards
 - flexibility
- **Some Linux distribution provide forks (like MariaDB)**
 - It's easy to reinstall MySQL over MariaDB
- **MySQL linux dependencies**
 - libaio
 - /lib64/libtinfo.so.5 (usually contained in ncurses-compat-libs)

Package Manager for MySQL on Linux

- Package managers simplify the MySQL installations, that also installs the dependencies
 - On RPM-based systems (Oracle Linux, Red Hat...) use yum install to install Community Edition
 - Example:

```
yum install mysql-community-server
```

```
yum install mysql-workbench
```
 - On APT-based systems (Ubuntu, Debian...) use apt-get install to install Community Edition
 - Example:

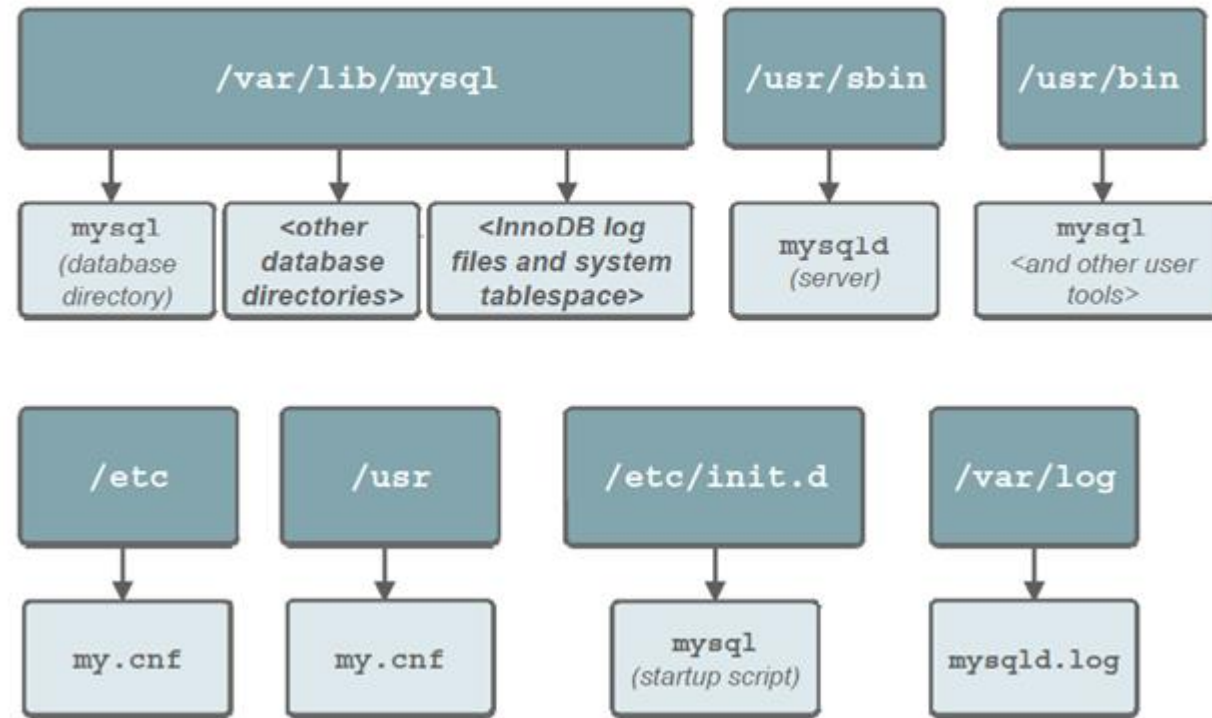
```
apt-get install mysql-community-server
```

```
apt-get install mysql-workbench
```
- Package managers have their own defaults
 - Where is the configuration files (/etc/my.cnf, /etc/mysqld/my.cnf, ...)?
- Enterprise Edition have no public repository, so download .rpm or .deb from <https://support.oracle.com> website
 - There is a dedicated package (MySQL Database/Components Yum Repository TAR) that helps setup of a self-hosted Yum repository

MySQL by Using a Package Manager on Linux

Example from Oracle Linux

Linux MySQL Server Installation Directories



MySQL User Accounts

- This is a short intro (details on accounts are in "Security" module)
- Within MySQL, accounts are composed by three elements
 - username@hostname + password
 - "hostname" is the reference of the machine where the mysql client run ('%' means "all the IPs")
- First installation create only the administrative account 'root'@'localhost'
 - Password is displayed on screen via warning message and/or stored in the error log file
 - Password have to be changed before any other command



Hands-On Labs



Lab 2a. Installation of MySQL Community

MySQL Installation deep dive (Linux)

- Basic steps are the same for all the platforms (we see here just Linux)
- We use here the generic binaries distribution
- MySQL installation steps
 - (optional) tune operating system settings
 - Create user that runs MySQL server process
 - Create directories infrastructure
 - First setup of configuration file (my.cnf)
 - Install & Initialize the instance
 - Start mysql server
 - Final actions
 - Create start/stop script
 - Tune configuration file

<https://dev.mysql.com/doc/refman/8.0/en/installing.html>

Tune operating system

- In many cases default OS configuration works fine
- If you need to open a lot of connections or create a lot of table increase limits for the mysql server user, here some docs
 - <https://support.oracle.com/epmos/faces/DocumentDisplay?id=2061003.1>
 - <https://support.oracle.com/epmos/faces/DocumentDisplay?id=1385400.1>
- Evaluate swap aggressivity (swappiness)
 - <https://support.oracle.com/epmos/faces/DocumentDisplay?id=1917687.1>
- NUMA support
 - https://dev.mysql.com/doc/refman/8.0/en/innodb-parameters.html#sysvar_innodb_numa_interleave
 - <https://support.oracle.com/epmos/faces/DocumentDisplay?id=2646142.1>
- Malloc
 - <https://support.oracle.com/epmos/faces/DocumentDisplay?id=1382963.1>
- And others in the manual and MOS

Create user that run MySQL server process

- It's a generic system user
- It's not required to have system privileges
- It's not required to have a shell
- Be careful with permissions for startup and backup&recovery
 - E.g. if you use sudo for backups

Create directories infrastructure

- There are only two mandatory variables: 'basedir' and 'datadir'
- Structure can be complex, for example

Folder	Usage
[...]/mysql	Root folder. Sometimes inside /usr/local/ or /opt/
[...]/mysql/mysql-X	Directory with binaries (basedir). Sometime referred with a symbolic link to simplify management
[...]/mysql/data	Directory with database files (datadir). In some linux distro is /var/lib/mysql/
[...]/mysql/logs	Directory where store logs. In some Linux distro is in /var/log/mysql/
[...]/mysql/temp	Directory for temporary files
[...]/mysql/uploads	Directory where server can read files to upload (LOAD DATA)
[...]/mysql/etc	Directory with configuration file(s) (my.cnf). In some Linux distro is in /etc/mysql
[...]/mysql/binlogs	Directory with binary logs used for replication and for backups

Balance the load on several disks

- **datadir (Random, SSD)**
 - Default location
- **innodb_data_file_path (Random, SSD)**
 - name, size, and attributes of InnoDB system tablespace data files
- **innodb_undo_directory (Random, SSD)**
 - path to the InnoDB undo tablespaces
- **innodb_log_group_home_dir (Sequential, spinning)**
 - path to the InnoDB redo log files
- **log-bin (Sequential, spinning)**
 - dual functionality: enable binary logging and set path/file name prefix
- **tmpdir (Random, SSD, tmpfs)**
 - Temporary files

First setup of configuration file

- my.cnf or my.ini (Windows)
- It's a text file split in sections
- It's read at startup
 - errors prevent server startup
- What is a good my.cnf?
 - Don't tune to much!
 - Windows has defaults
 - There are some guidelines...

```
[mysqld]
# General
port                =3307
server_id           =123456
socket              =/mysql/temp/mysql.sock

# File locations
basedir             =/mysql/8014
plugin-dir          =/mysql/8014/lib/plugin
datadir             =/mysql/data
tmpdir              =/mysql/temp
log-error           =/mysql/log/err_log.log
general_log_file    =/u01/log/general_log.log
slow_query_log_file =/mysql/log/slow_log.log
innodb-log-group-home-dir=/mysql/translog
secure-file-priv    =/mysql/uploads

# Maximum limits
max-connections     =200

# InnoDB
innodb_buffer_pool_size =512M
innodb_flush_method  =O_DIRECT

# MyISAM
key_buffer_size     =128M
#Other settings
sql_mode            =NO_ENGINE_SUBSTITUTION,STRICT_TRANS_TABLES

# Threadpool plugin
plugin-load=thread_pool.so

...
```

Socket connection

Not /mysql/data/bin!

Some important settings in my.cnf

- **port**, default 3306
 - unique inside the server
- **server_id**, default 0 (≤ 8.0) or 1
 - unique for each instance for all the datacenters
- **innodb_dedicated_server**
 - Set automatically most critical variables
- **innodb_buffer_pool_size**, default 128M
 - very important to increase to manage the online dataset in memory (thumb rule: 70-80% of RAM)
- **innodb_log_file_size**, default 50M
(**deprecated** since 8.0.30)
 - Redo logs, probably need to increase
- **innodb_redo_log_capacity**, default 100MB
(**new** 8.0.30+)
 - Redo logs, probably need to increase
- **innodb_flush_log_at_trx_commit**,
 - decide "Durability", use default if you care about your data
- **secure_file_priv**
 - used to limit the effect of data import and export operations
- **max-connections**, default 151
 - probably not enough
- **open-files-limit**, default 5000
 - it may require to be increased
- **sql_mode**, (empty before 5.6)
 - affect the SQL syntax and the data validation checks MySQL performs
- **transaction-isolation** (REPEATABLE READ)
 - consider READ-COMMITTED
- **skip_name_resolve**, default OFF
 - performances, but also behavior of "localhost"
- **log_timestamps = SYSTEM**
 - how the timestamps are recorded inside logs

New redo logs management (8.0.30+)

- Prior to MySQL 8.0.30, InnoDB creates two redo log files named `ib_logfile0` and `ib_logfile1`
 - Modifying redo log capacity requires to change `innodb_log_files_in_group` and `innodb_log_file_size` variables (now deprecated) and restart the instance
- From 8.0.30 InnoDB now supports dynamic configuration of redo log capacity
 - The `innodb_redo_log_capacity` system variable can be set at runtime **to increase or decrease** the total amount of disk space occupied by redo log files
 - InnoDB maintains 32 redo log files in the `#innodb_redo` directory (`innodb_redo_log_capacity / 32`)
- Calculating the optimal size (during your peak):
 - get an estimation of the required amount for the Redo Log. Here an example of estimation

```
SELECT VARIABLE_VALUE FROM performance_schema.global_status WHERE VARIABLE_NAME='Innodb_redo_log_current_lsn' INTO @a; SELECT SLEEP(60) INTO @garb; SELECT VARIABLE_VALUE FROM performance_schema.global_status WHERE VARIABLE_NAME='Innodb_redo_log_current_lsn' INTO @b; SELECT ROUND(ABS(@a - @b)/1024/1024,2) AS MB_per_min, ROUND(ABS(@a - @b)/1024/1024*60,2) AS MB_per_hour;
```
 - rule of thumb is to make the Redo Logs big enough to hold at most 1h of logs

'#ib_redo582'	'#ib_redo590'	'#ib_redo598'	'#ib_redo606_tmp'
'#ib_redo583'	'#ib_redo591'	'#ib_redo599'	'#ib_redo607_tmp'
'#ib_redo584'	'#ib_redo592'	'#ib_redo600'	'#ib_redo608_tmp'
'#ib_redo585'	'#ib_redo593'	'#ib_redo601'	'#ib_redo609_tmp'
'#ib_redo586'	'#ib_redo594'	'#ib_redo602'	'#ib_redo610_tmp'
'#ib_redo587'	'#ib_redo595'	'#ib_redo603_tmp'	'#ib_redo611_tmp'
'#ib_redo588'	'#ib_redo596'	'#ib_redo604_tmp'	'#ib_redo612_tmp'
'#ib_redo589'	'#ib_redo597'	'#ib_redo605_tmp'	'#ib_redo613_tmp'

Global Variables

- Global (system) variables can be set at different levels
 - As option when start mysql server
 - Configuration file (my.cnf)
 - Temporarily within the client (set global variable_name=value)
 - Permanently within the client (set persist variable_name=value)
 - At boot time it's read after my.cnf
- Persistent runtime configuration changes
 - Any GLOBAL DYNAMIC configuration variable can be modified
 - The settings will be persisted to a file named mysqld-auto.cnf (JSON format)
 - New table performance_schema.persisted_variables
 - New table performance_schema.variables_info

VARIABLE_NAME	VARIABLE_SOURCE	VARIABLE_PATH	MIN_VALUE	MAX_VALUE
foreign_key_checks	DYNAMIC		0	0
innodb_buffer_pool_size	GLOBAL	C:\my.ini	5242880	9223372036854775807
innodb_log_file_size	GLOBAL	C:\my.ini	4194304	18446744073709551615
log_syslog	COMMAND_LINE		0	0
long_query_time	PERSISTED	C:\mysql-trunk\data\mysqld-auto.cnf	0	4719230590769954816
max_connections	PERSISTED	C:\mysql-trunk\data\mysqld-auto.cnf	1	100000

<https://dev.mysql.com/doc/refman/8.0/en/server-option-variable-reference.html>
<https://dev.mysql.com/doc/refman/8.0/en/persisted-system-variables.html>

Initialize the Data Directory

- The initialization create the data dictionary, redo logs, undo logs... and all the required files to run the instance
- After MySQL is installed, the data directory must be initialized
 1. Change location to the top-level directory of your MySQL installation
 2. Grant directory user and group ownership to your 'mysql' user and 'mysql' group
 3. initialize the data directory, e.g.

```
$> bin/mysqld --initialize --user=mysql
```
- (Optional) With `--initialize-insecure`, no root password is generated
 - This is insecure; it is assumed that you intend to **assign a password** to the account in a timely fashion **before putting the server into production use**

Final actions

- Choose how to start the mysqld service

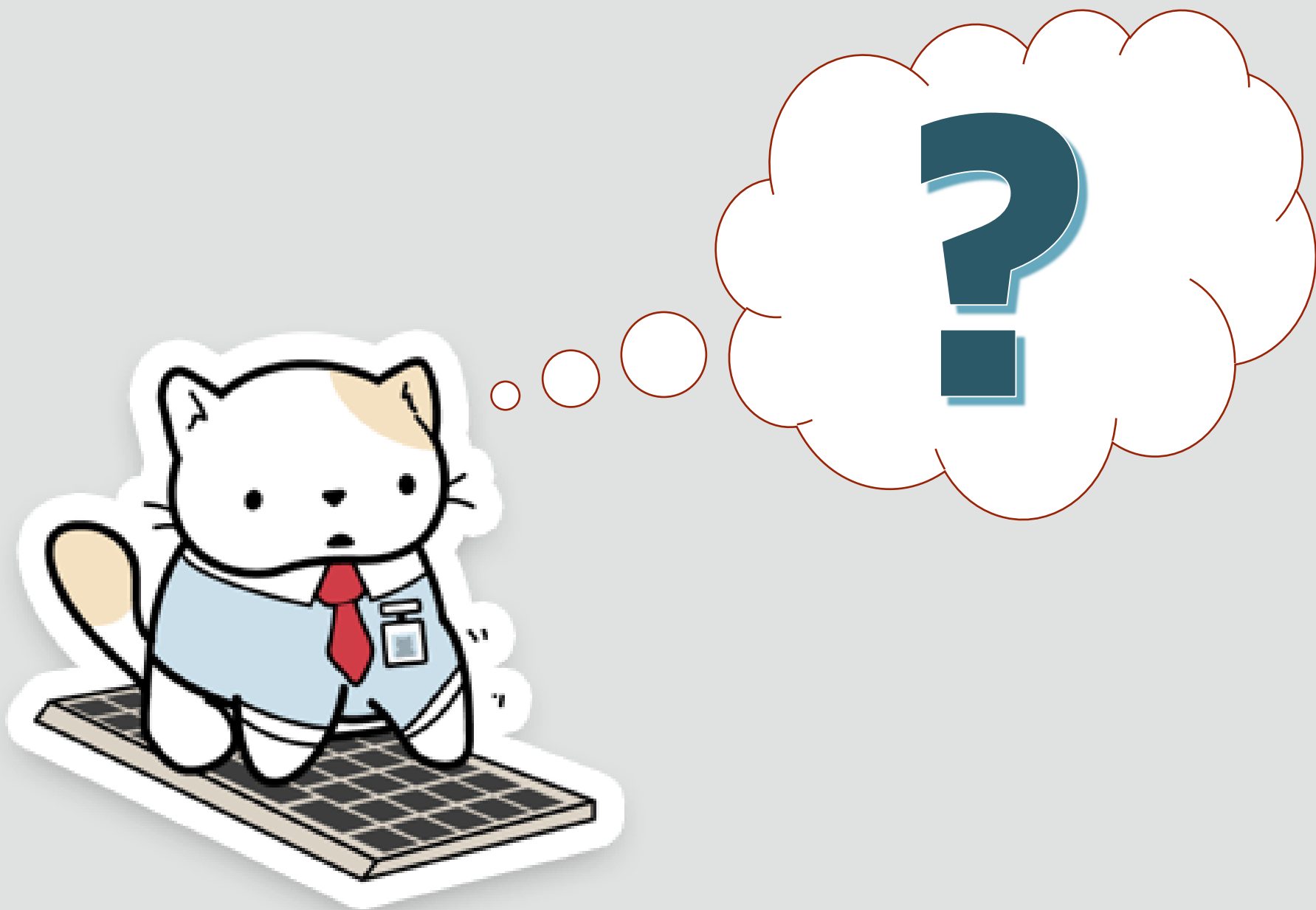
- to automatize startup/shutdown/respawn
- mysqld or mysqld_safe, systemd, windows service...

```
--mysqld -help and mysqld -help -verbose [check for orders!]  
--initialize  
--defaults-file=/mysql/my.cnf  
--bind-address  
--skip-grant-tables
```

- Refine my.cnf/my.ini, for example to

- reduce database creation time you may disable binary logs
- because you are using a default configuration just for creation time
- application settings
- etc.

<https://dev.mysql.com/doc/refman/8.0/en/automatic-start.html>



Hands-On Labs



2b. Detailed Installation of MySQL on Linux

2c. Verify the new MySQL Installation on Linux and import test databases

MySQL Upgrades and Downgrades

- Upgrades and Downgrades are offline operations
- It is best practice to have a full backup of your MySQL environment before (Trigger, Stored Procedures, configs, binary etc.)
- Options
 - **In-Place Upgrade:** Involves shutting down the old MySQL version, replacing the old MySQL binaries or packages with the new ones, restarting MySQL on the existing data director
 - Before 8.0.16 run `mysql_upgrade` to complete the upgrade process
 - **Logical Upgrade:** Involves exporting existing data from the old MySQL version using `mysqldump/MySQL Shell`, installing the new MySQL version, loading the dump file into the new MySQL version, and running `mysql_upgrade`
- **Don't forget MySQL Connectors and libraries as well**
 - you may need to recompile any C or C++ programs that link statically to the “.lib” or “.a” forms of MySQL libraries
- **Replication may help to reduce application downtime**

MySQL Upgrades

Releases before LTS/Innovation releases

- For In-Place upgrades, upgrading is supported only one major release level at a time
 - Example from 5.5 → 5.6 → 5.7 → 8.0 etc.
 - Logical Upgrade that skips major releases is not supported
- Upgrading from a release series version to a newer release series version is supported, for example, upgrading from 8.0.17 to 8.0.28 is supported
 - Usually an In-Place Upgrade
- Downgrades within 8.0 requires dump&load

MySQL Upgrades/Downgrades

LTS releases

- In-place upgrades from LTS release is supported only to the next LTS release
 - Example: 8.4 → 9.7 → 10.7 etc.
 - Logical Upgrade that skips major releases is not supported
- Upgrading/downgrading from a minor to a newer version is supported within the same major
 - Example: LTS 8.4.11 → LTS 8.4.20
 - Usually an In-Place Upgrade
- In-place upgrades from LTS release to a future Innovation release will be supported
- Downgrade from Innovation to LTS (rollback purposes only) requires dump&load

<https://blogs.oracle.com/mysql/post/introducing-mysql-innovation-and-longterm-support-lts-versions>

MySQL Upgrades/Downgrades

Innovation releases

- In-place upgrades from one Innovation to a future Innovation release will be supported, up until the next LTS release
 - Example: 8.1 → 8.2 → 8.3 → 8.4 → 9.0.0 → 9.7.0 etc.
 - Logical Upgrade that skips major releases is not supported
- Upgrading/downgrading from a minor to a newer version is supported, up until the next LTS release
 - Example: 8.1 → 8.3
 - Usually an In-Place Upgrade
- In-place upgrades from Innovation release to a future LTS release will be supported
- Downgrade from LTS to Innovation (rollback purposes only) requires dump&load

<https://blogs.oracle.com/mysql/post/introducing-mysql-innovation-and-longterm-support-lts-versions>

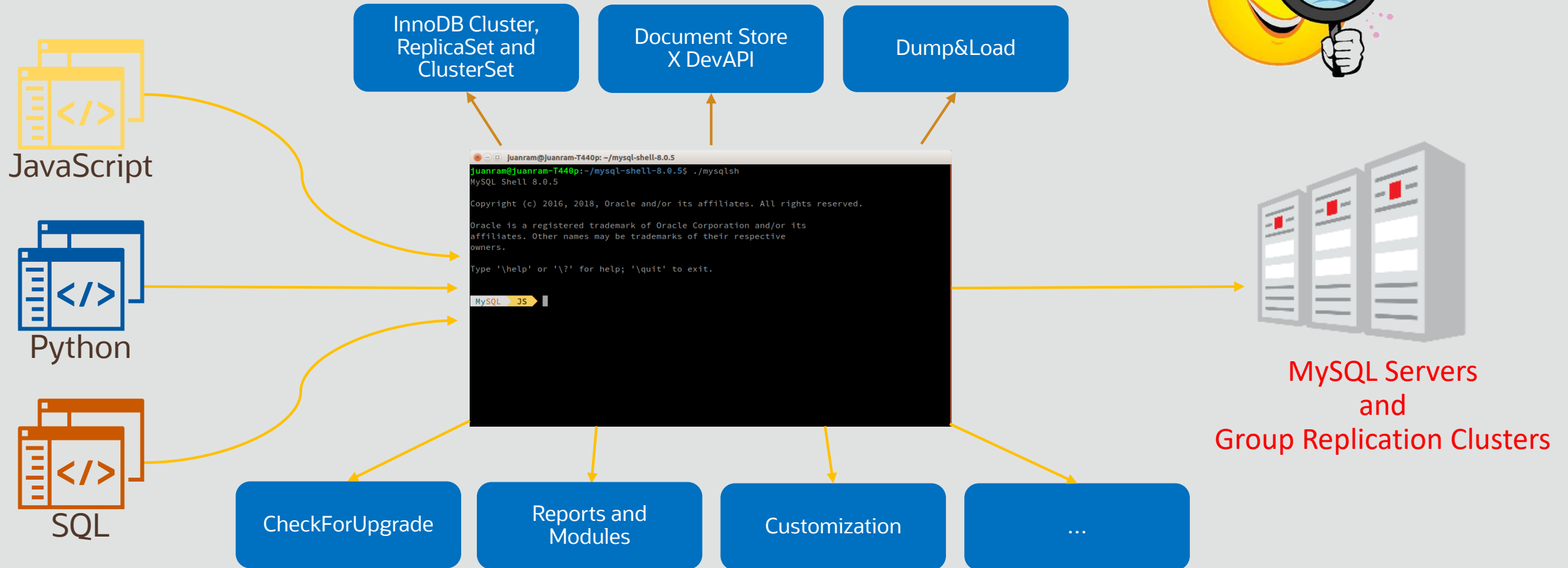
From Community to Enterprise

- Changing from MySQL Community to MySQL Enterprise Edition is just binaries substitution
 1. Download the Enterprise software, exactly same version as community
 2. Stop the community binaries
 3. Remove the community binaries
 4. Install the Enterprise binaries
 5. Start the instance using the same configurations files, datafiles, etc. (no changes)

MySQL Shell 8.0

A single unified client for all administrative and operations tasks

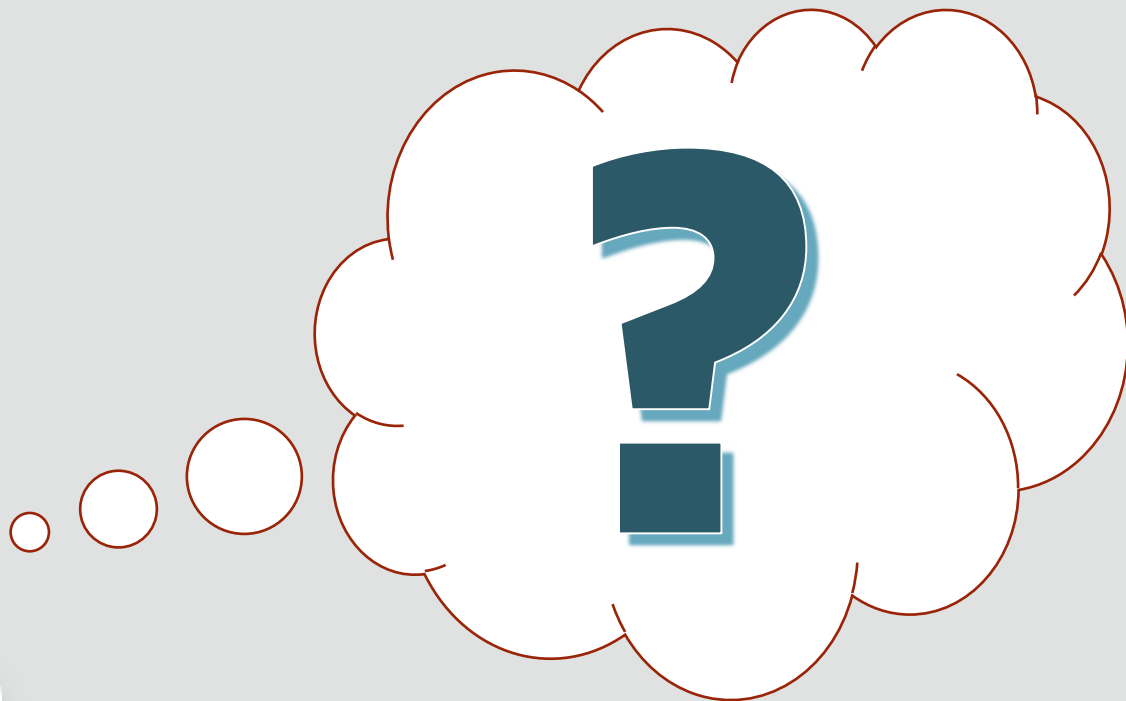
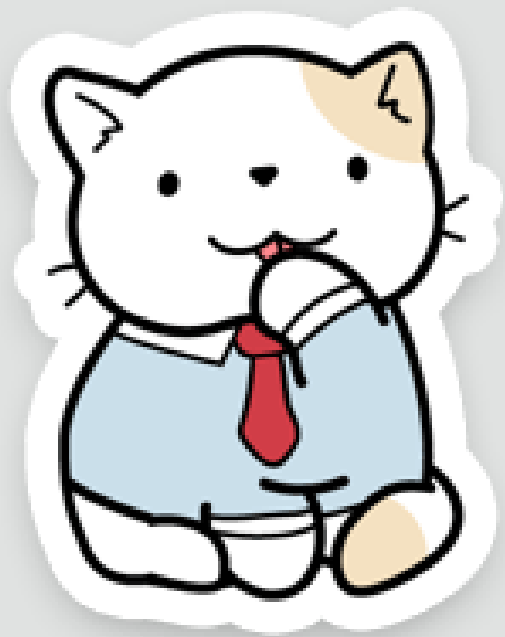
<https://dev.mysql.com/doc/mysql-shell/8.0/en/>



MySQL Shell Upgrade Checker

- Using MySQL Shell we can check the status of the server to be upgraded
- Run MySQL Shell before upgrade:

```
MySQL JS > util.checkForServerUpgrade("user@127.0.0.1:3306", {"targetVersion":"8.0.19",  
"configPath":"/etc/my.cnf"})
```
- MySQL Shell will identify if you are using any reserved keywords, removed datatypes or if some parameters in your configuration have new defaults and much more
 - Included my.cnf/my.ini
- Upgrade checker will not identify problems like GIS changes or deprecated MySQL functions used by your application
- From MySQL Shell 8.0.21, the user account requires RELOAD, PROCESS, and SELECT privileges



Hands-On Labs



2d. MySQL Shell: the new client



ORACLE